

VegShift — Detailed Pipeline Guide (Steps 0–9)

Comprehensive step-by-step breakdown with example inputs, outputs, commands and notes. Shareable PDF for teammates.

Contents

1. Step 0 — Master Index
2. Step 0b — Preprocess raw climate data
3. Step 1 — Koppen-Geiger classification
4. Step 1b — Transition detection
5. Step 2 — Climate feature aggregation
6. Step 3 — Groundwater aggregation
7. Step 4 — FAO GAEZ extraction
8. Step 5 — Join + Features + CVLE labels
9. Step 6 — TFT Training
10. Step 7 — TFT Predict + Attention
11. Step 8 — Baselines
12. Step 9 — SHAP Explainability

1. Step 0 — Master Index

Builds the 10-city × 25-year backbone table used to join all datasets.

Command: `python pipeline/step0_master_index.py`

Inputs:

- (hardcoded) cities list and years 2000–2024

Outputs:

File	Description / Example
master_index.csv	city,year Delhi,2000 Mumbai,2000

Notes:

Produces data/processed/master_index.csv (250 rows).

2. Step 0b — Preprocess raw climate data

Standardizes daily climate fields; derives humidity where missing; unit conversions.

Command: `python pipeline/step0b_preprocess_datasets.py`

Inputs:

- Raw daily CSV: city,date,temperature_2m_max,temperature_2m_min,precipitation_sum

Sample input row:

Delhi,2000-01-01,32.5,21.1,0.0

Outputs:

File	Description / Example
kaggle_climate.csv	city,date,temp_max,temp_min,temp_mean,rainfall

Sample output row:

Delhi,2000-01-01,32.5,21.1,26.8,0.0

Notes:

Also computes `humidity` using apparent-temperature inversion when missing.

3. Step 1 — Koppen-Geiger classification

Aggregates daily to annual and assigns a Koppen zone per city-year.

Command: `python pipeline/step1_koppen_classification.py`

Inputs:

- Annual aggregates: T_ann, P_ann, T_min_month, T_max_month, monthly rainfall series

Outputs:

File	Description / Example
koppen_annual.csv	city,year,koppen_zone,koppen_zone_enc

Sample output row:

Delhi,2005,BSh,2

4. Step 1b — Transition detection

Detects persistent changes in Koppen zone (persist ≥ 3 years).

Command: `python pipeline/step1b_transition_detection.py`

Inputs:

- `koppen_annual.csv` (time series for each city)

Outputs:

File	Description / Example
<code>transition_report.json</code>	<code>[[{city,transition_year,from_zone,to_zone}]]</code>

Sample output row:

```
{'city':'Lucknow','transition_year':2007,'from_zone':'Cwa','to_zone':'BSh'}
```

5. Step 2 — Climate feature aggregation

Computes crop-relevant annual features: monsoon onset, GDD, sowing-window miss, water deficit.

Command: `python pipeline/step2_climate_aggregate.py`

Inputs:

- `kaggle_climate.csv` daily rows for a city-year

Outputs:

File	Description / Example
<code>climate_annual.csv</code>	<code>city,year,temp_mean,rainfall_annual,gdd_accumulation,sowing_w</code>

Sample output row:

Lucknow,2005, temp_mean:25.42, rainfall_annual:823.4, gdd_accumulation:3729.85,
sowing_window_miss:1.0

6. Step 3 — Groundwater aggregation

Samples nearest CGWB wells (50 km) to compute pre/post monsoon depth, depletion, recharge_efficiency.

Command: `python pipeline/step3_groundwater_aggregate.py`

Inputs:

- CGWB CSV with season columns (Jan-YY, May-YY, Aug-YY, Nov-YY)

Outputs:

File	Description / Example
groundwater_annual.csv	city,year,pre_monsoon_depth_mbgl,post_monsoon_depth_mbgl,c

Sample output row:

Ahmedabad,2005, pre_monsoon_depth_mbgl:7.98, post_monsoon_depth_mbgl:2.075,
depletion_rate: -1.17

Notes:

Jaipur uses a nearest-well fallback and is flagged with gw_imputed=2.

7. Step 4 — FAO GAEZ extraction

Samples raster suitability for the city coordinate and maps to ECOCROP thresholds.

Command: `python pipeline/step4_gaez_extract.py`

Inputs:

- GAEZ GeoTIFF per crop

Outputs:

File	Description / Example
gaez_baseline.csv	city,gaez_baseline_class,gdd_min,water_req

Sample output row:

Lucknow, gaez_baseline_class:7 (clipped from 8), gdd_min:2500, water_req:1500

8. Step 5 — Join + Features + CVLE labels

Joins processed tables, imputes missing values, creates compound features and the CVLE label.

Command: `python pipeline/step5_join_and_features.py`

Inputs:

- master_index + climate_annual + groundwater_annual + gaez_baseline + koppen_annual

Outputs:

File	Description / Example
vegshift_master.csv	master table with features + cvle_label

Sample output row:

Lucknow, 2005, dual_deficit:1, gdd_adequate:1, cvle_label:1

Notes:

CVLE rule: consecutive dual_deficit years + at least 2 of 3 thresholds (sow_miss>0.6, water_deficit>0.4, gdd_inadequate).

9. Step 6 — TFT Training

Prepares TimeSeriesDataSet (encoder_len=5) and trains a Temporal Fusion Transformer; saves best checkpoint.

Command: `python pipeline/step6_tft_train.py`

Inputs:

- `vegshift_master.csv` with ``time_idx`` per city

Outputs:

File	Description / Example
<code>models/tft/vegshift-tft-best-VERSION.ckpt</code>	trained checkpoint

Notes:

Uses quantile loss and outputs a quantile ensemble (7 quantiles).

10. Step 7 — TFT Predict + Attention

Loads best checkpoint, predicts CVLE probabilities for validation windows, extracts attention weights.

Command: `python pipeline/step7_tft_predict.py --checkpoint models/tft/vegshift-tft-best.ckpt`

Inputs:

- `vegshift_master.csv` + checkpoint

Outputs:

File	Description / Example
<code>tft_predictions.csv</code>	<code>city,year,predicted_cvle_score</code>
<code>tft_attention_weights.json</code>	<code>city:[5-lag weights]</code>

Sample output row:

Delhi, 2021, 0.0096

Sample attention:

Lucknow: [0.20, 0.21, 0.15, 0.22, 0.23]

11. Step 8 — Baselines

Trains Random Forest, Logistic Regression, and an LSTM sequence baseline; saves models and metrics.

Command: `python pipeline/step8_baselines.py`

Inputs:

- `vegshift_master.csv`

Outputs:

File	Description / Example
<code>models/baselines/*.pkl, *.pt</code>	<code>data/output/baseline_metrics.json</code>

Example metrics:

RF AUC:1.0, LR AUC:1.0, LSTM AUC:0.846 (test split small/imbalanced)

12. Step 9 — SHAP Explainability

Computes SHAP values for the RF baseline and exports global + per-city feature importance.

Command: `python pipeline/step9_shap_explainability.py`

Inputs:

- `vegshift_master.csv`
- `models/baselines/rf_baseline.pkl`
- `scaler.pkl`

Outputs:

File	Description / Example
<code>data/output/shap_explanation.json</code>	global and per-city importance lists

Sample output row:

global_importance top: `['recharge_efficiency', 'crop_water_deficit', 'gdd_accumulation']`

Verification Checklist

- data/processed/master_index.csv present (250 rows)
- data/processed/kaggle_climate.csv present and daily
- data/processed/vegshift_master.csv includes `cvle\_label`
- models/tft contains at least one best checkpoint
- models/baselines contains RF/LR/LSTM and data/output/baseline_metrics.json

Evaluation notes

- The dataset is imbalanced; prefer PR-AUC and aggregated evaluation over multiple years or cross-validation.
- Calibrate model probabilities before picking operational thresholds.

Contact: pipeline owner for dataset provenance or to request extended evaluation windows.